CSC 251 Lecture 18 - Machine Learning (Part 2)

Process of Machine Learning (review)

Generally, we will complete the following steps:
1) Choose a class of model and import the appropriate class from package.

2) Choose model parameters and instantiate the class.

3) Arrange data into a features matrix and target vector

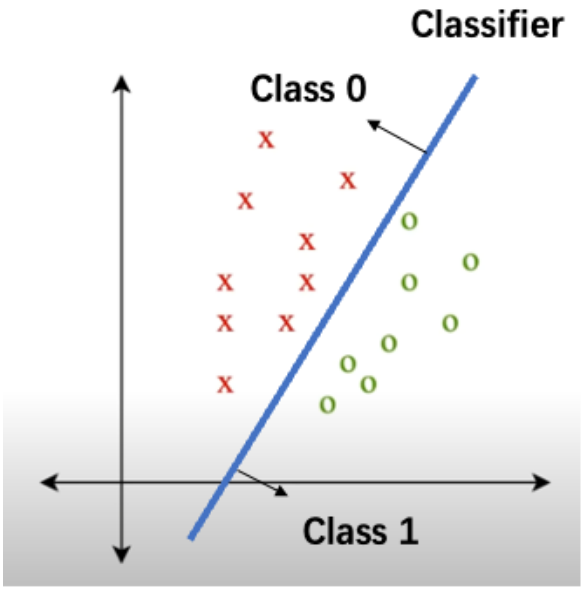
4) Fit the model to your data by calling the fit methods of the model instance

5) Apply the model to new data
   a) Predict labels for supervised learning
   b) Transform or infer properties for unsupervised learning

6) Evaluate the model and adjust the hyperparameters

Supervised Learning Classifiers

Classification – given a set of data points and their corresponding labels, "learn" how they are classified so that a new point can be put in the correct class. Find the best decision boundary.



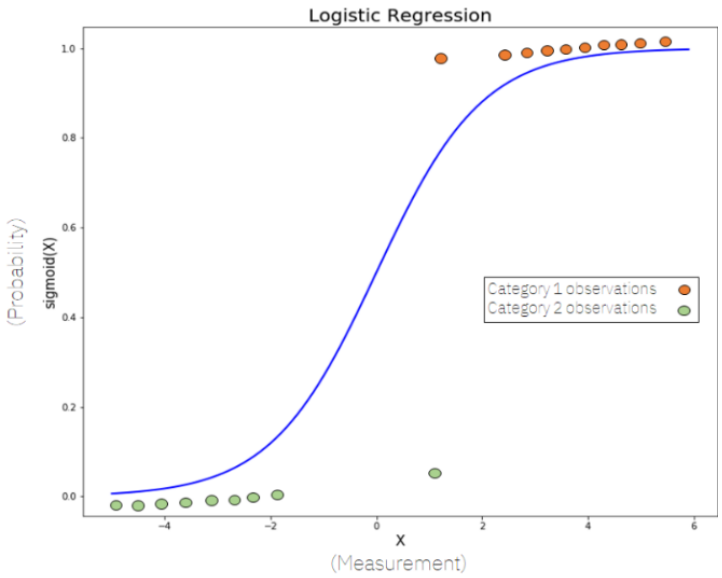
A few common classification algorithms:

- linear regression
- logistic regression
- kNN (k-nearest neighbors)
- SVM (Support Vector Machine)
- Naïve Bayes
- Random Forest
- K-means Clustering
- Decision Tree

Logistic Regression

Linear regression (Ordinary Least Squares) is appropriate for numeric values.
What about categorical variables?

What if our x variables (predictors) are continuous but our y variable is categorical?
Logistic regression (Maximum Likelihood Estimation) uses a sigmoid function which puts the probability between 0 and 1. We can declare a class or label depending upon which side of 0.5 the value lies.



$$y = \frac{1}{1 + e^{-kx}}$$

https://builtin.com/machine-learning/logistic-regression-classification-algorithm

Multiple predictors – uses logit function for each predictor variable rather than for the dependent variable. The algorithm determines the probability that the data belongs to a particular class based upon the predictor variables.

Designated as “binary”, “multinomial”, or “ordinal” logistic regression based upon the number and type of categorical variables.

For the iris example from last time, after splitting the data into testing and training sets:

```
from sklearn.linear_model import LogisticRegression
model2 = LogisticRegression(multi_class='multinomial', max_iter= 200).fit(Xtrain, ytrain)
pred2 = model2.predict(Xtest)
accuracy_score(ytest, pred2)
```

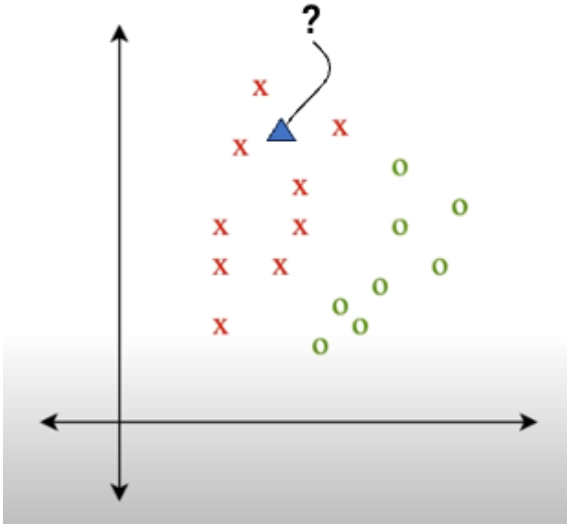
```
0.9736842105263158
```

How does this compare with the linear regression model?

k-Nearest Neighbors (kNN)

kNN – Compare a new data point to data points with correct class labels and find the `k` most similar neighboring points. The majority vote provides the winning label/class for the new point.

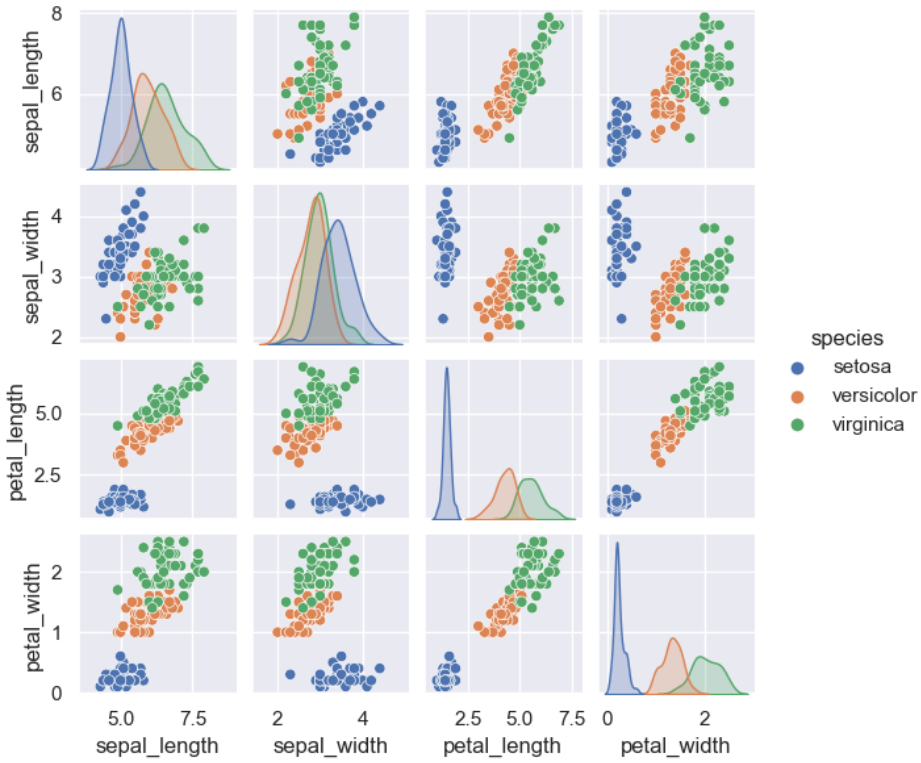
“If it walks like a duck and quacks like a duck, then it must be a duck!”

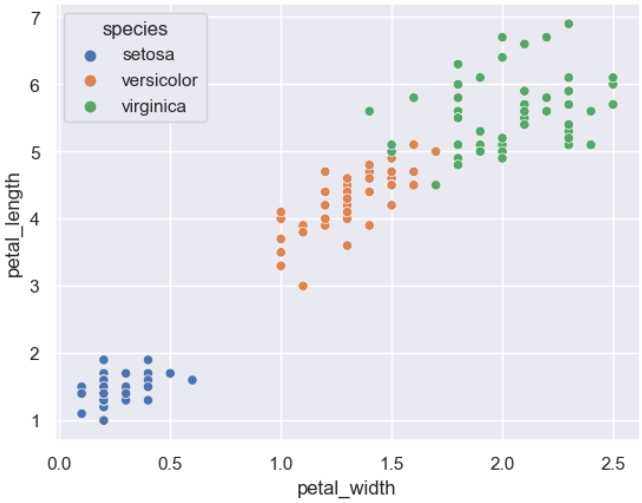


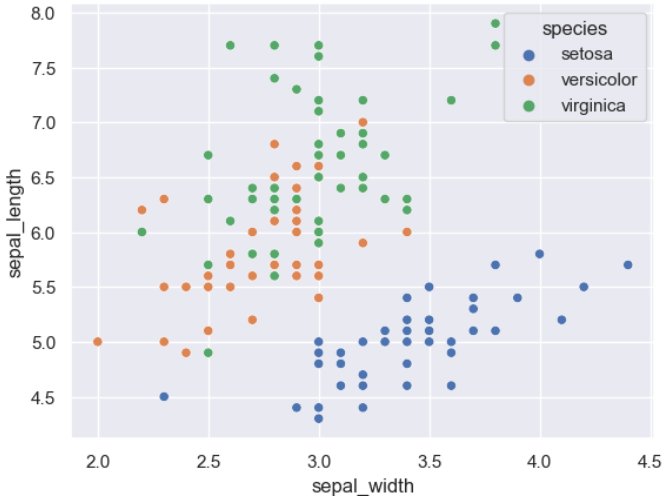
How do you determine “nearest”?

Nearest neighbor activity based upon academic classification using k = 2, k = 4, k = 8
Results?

Recall the Iris dataset:







What should k be? (usually between 2 and 20)
What if k is too small? overfit to training data
What if k is too large? underfit

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(Xtrain, ytrain)
y_pred = knn.predict(Xtest)
accuracy_score(ytest, y_pred)
```

```
0.9666666666666667
```

How good was the classification? Also check the confusion matrix:

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Sources: [3][4][5][6][7][8][9][10]

$$Precision = \frac{TP}{TP + FP}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Recall/Sensitivity = \frac{TP}{TP + FN}$$

```
from sklearn.metrics import confusion_matrix
confusion_matrix(ytest, y_pred)
```

```
array([[10,  0,  0],
       [ 0,  9,  1],
       [ 0,  0, 10]])
```

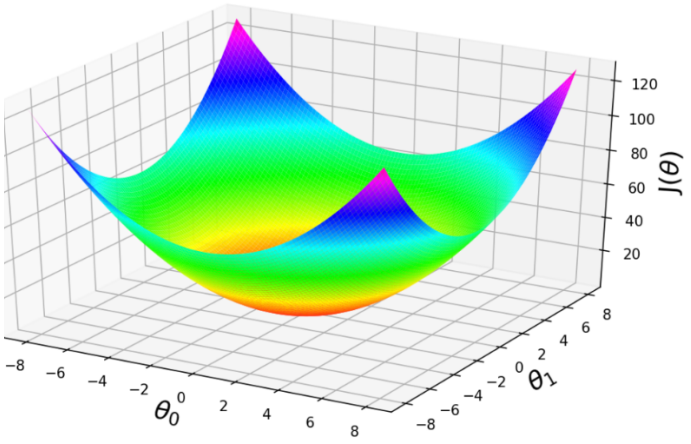
Increase the k value to 5:

```
array([[13,  0,  0],
       [ 0, 16,  0],
       [ 0,  0,  9]])
```

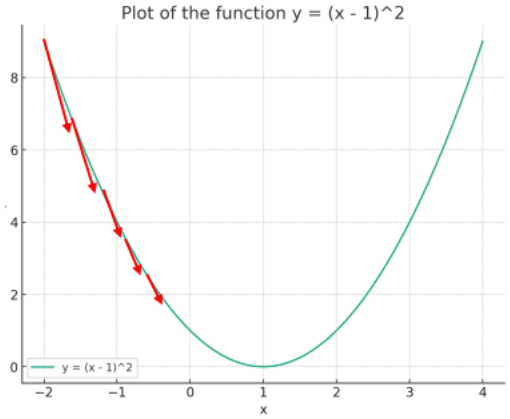
```
1.0000000000000000
```

Optimization - Gradient Descent

Computes a slope down a loss function L(x, y) or error surface in order to find a model that provides the best fit for the given data. Given the following error surface, the height of the function indicates the amount of error for a given (x, y) or (m, b) point. Some lines or (m, b) points yield a smaller error.



In 2D, if our error function is $L(x) = (x-1)^2$



We want to move "downhill" to minimize the value of L(x) so we compute the negative gradient:

$$-\nabla L(x) =$$

The best fit line will have a gradient of zero, at the minimum of our graph.

Extend this to 3D…

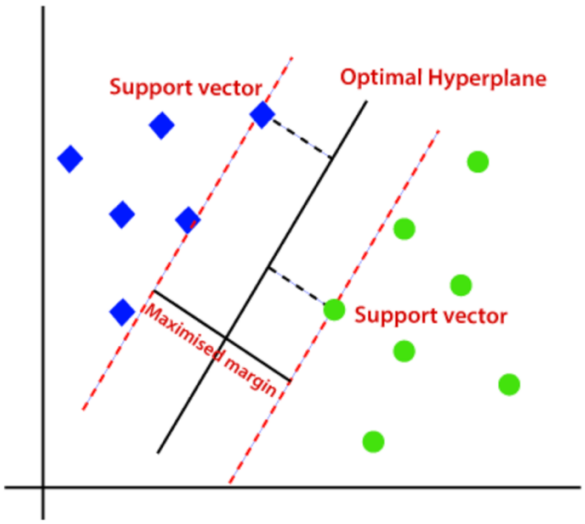
SVM (Support Vector Machines)

- Searches for the optimal linear separating hyperplane as a decision boundary
- Creates a margin rather than a zero-width line between the classes
- Best with smaller datasets

hyperplane – one dimension less than the ambient space (a line in 2D, a plane in 3D space, …)

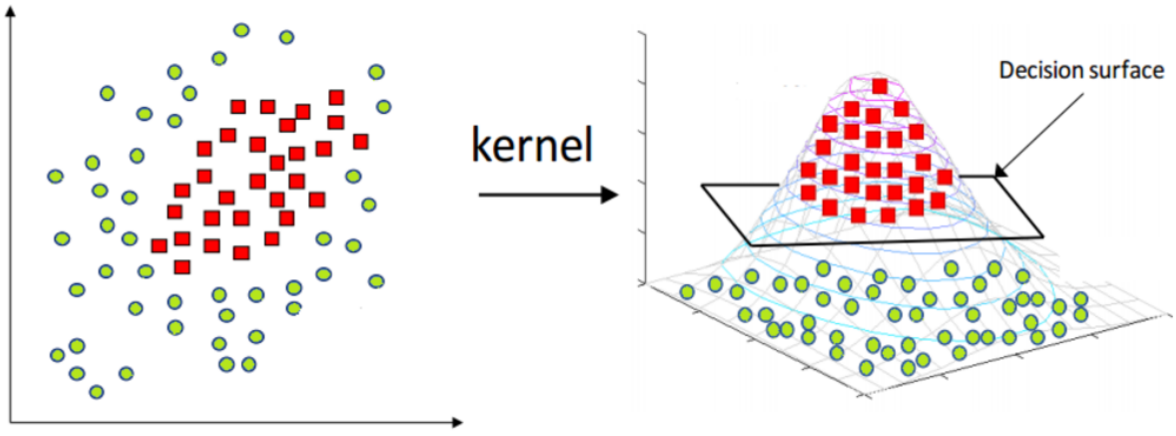
support vectors – the points closest to the hyperplane used to define a separating line

margin – the distance between the hyperplane and the support vectors, larger is better.

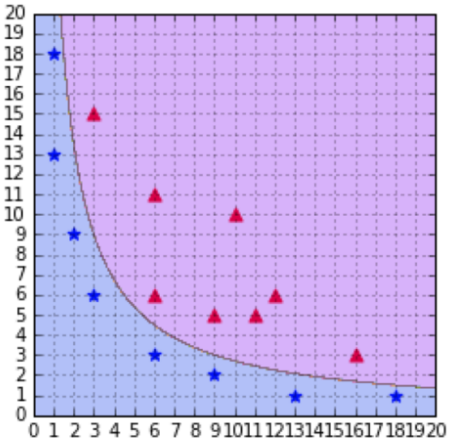


https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/

Tricks for non-linear data – pick a different kernel (function) and tune its parameters

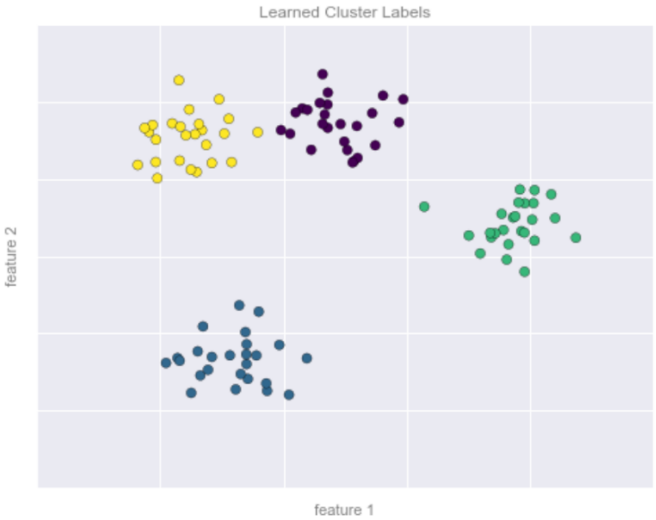


Non-linear decision boundary:

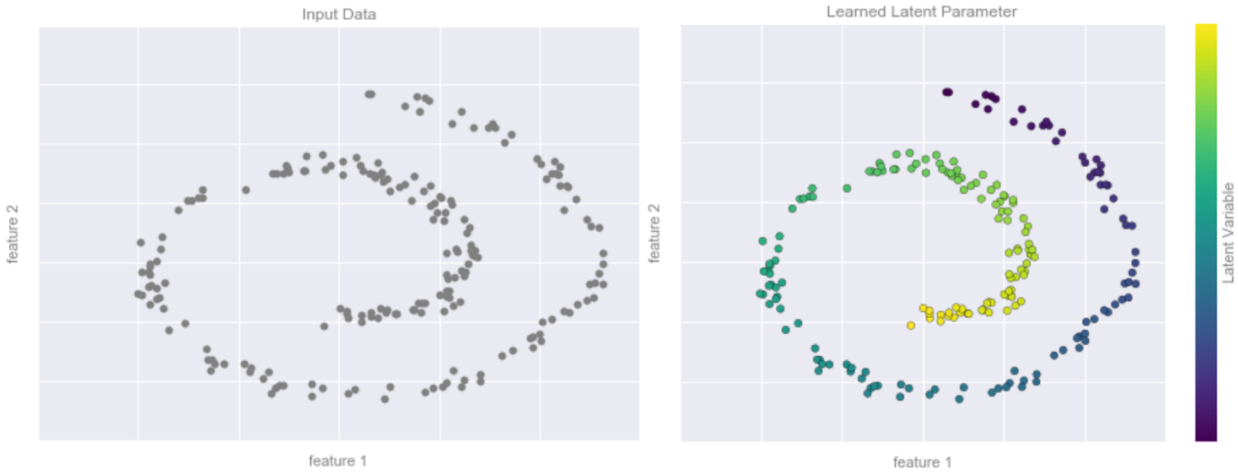


Unsupervised Learning

Data points are not labeled, so the goal is to organize the data or describe its structure.
Clustering – assigning the data into subsets that are similar in some way.
k-means: choose how many clusters (k) that are desired and determine the centroid of the clusters



Dimensionality Reduction – reduce the size of the dataset by eliminating independent variables that do not influence the dependent variable, one technique is Principal Component Analysis (PCA).



Back to our iris dataset, let's use PCA for Dimensionality Reduction from the 4 features to 2.

```
from sklearn.decomposition import PCA
model = PCA(n_components=2)
```

Notice y is not used for the fit!

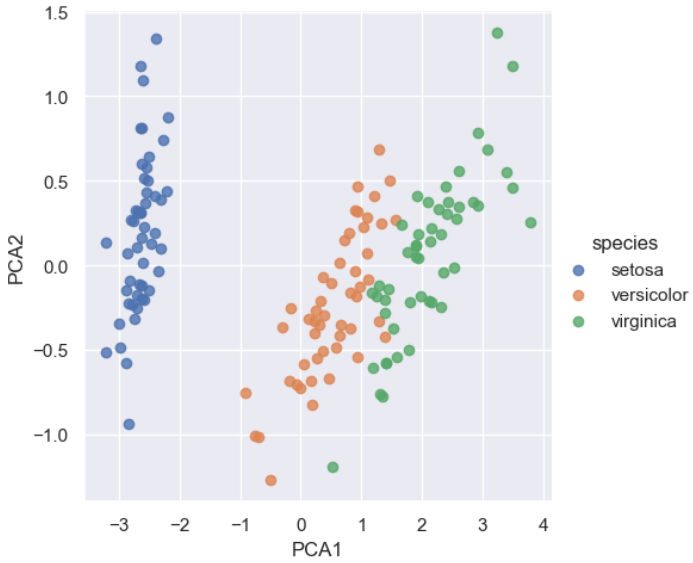
```
model.fit(X_iris)
```

Transform the data to 2 dimensions (features):

```
X_2D = model.transform(X_iris)
```

Plot by inserting the results back into the original iris DataFrame:

```
iris['PCA1'] = X_2D[:, 0]
iris['PCA2'] = X_2D[:, 1]
sns.lmplot(x = "PCA1", y= "PCA2", hue='species', data=iris, fit_reg=False)
```



Dimensionality Reduction can also be used with hand-written digits, to reduce to a 2D representation:



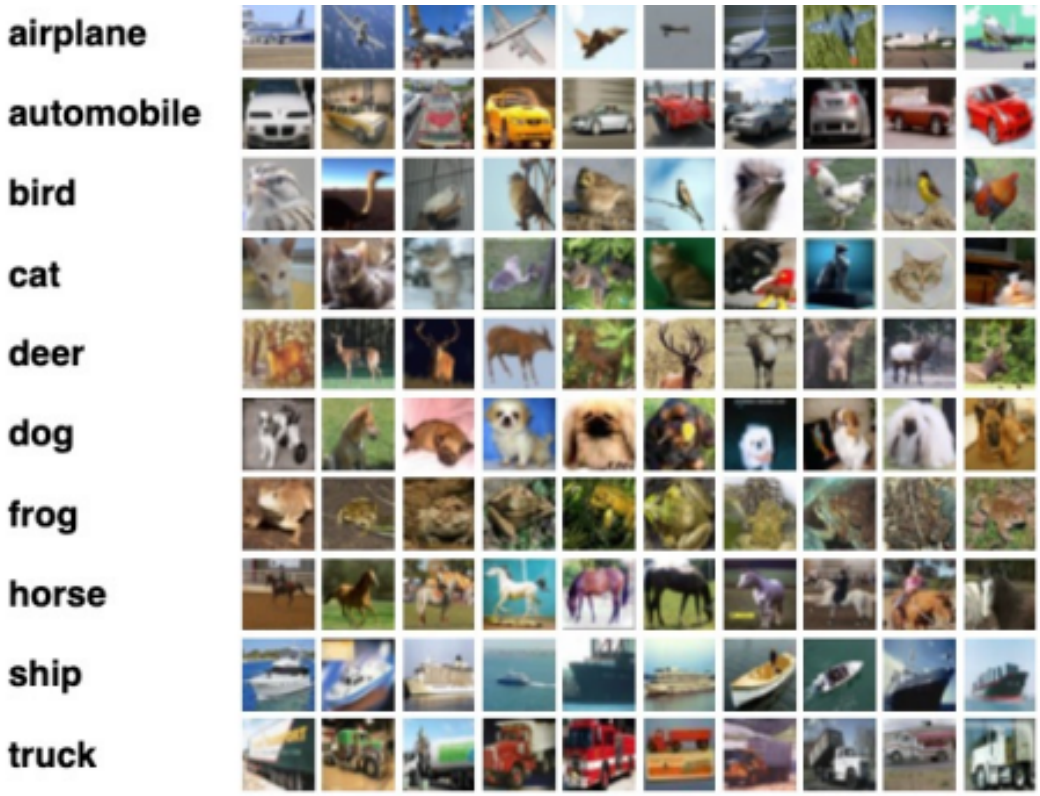
Common Machine Learning Datasets

Fisher Iris flower data from 150 samples to classify iris species based upon sepal length & width and petal length & width

MNIST (Modified National Institute of Standards and Technology) image database of 70K hand-written digits



CIFAR-10 (Canadian Institute For Advanced Research) image database with 60K 32 x 32 color images in 10 classes used to train ML and computer vision algorithms



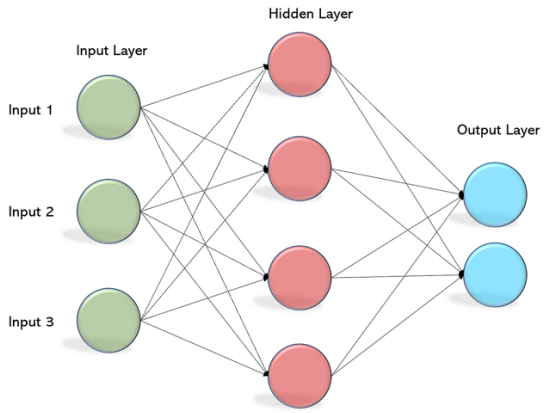
Deep Learning

Scikit-learn – easy to use for small datasets with traditional machine learning algorithms

PyTorch – Pythonic, object-oriented, better suited for deep learning, good GPU acceleration for parallel processing

TensorFlow – Optimized for deep learning, large-scale deployments with large datasets

Deep learning constructs complex models by stacking layers, creating neural networks, each layer passes its output to the next layer so that the network learns through iteration, improving its predictions and self-correcting by adjusting internal parameters. An epoch is one complete pass through the training dataset.



Types of deep learning models include Feedforward Artificial Neural Networks (ANNs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs), and Residual Network (ResNet) architecture.

Steps included (see notebook):

1) Import torch libraries
2) Select the device type (CUDA versus CPU)
3) Load dataset
4) Configure model and train model, tune parameters
5) Test and evaluate model, compare with model provided
6) Deploy model with new data to make predictions

Residual Network (ResNet) architecture creates residual blocks of convolutional layers, skipping connections between layers, often used for computer vision.

Lecture 18 Activity:

Download notebook and `pretrained_resnet110.pth` files, upload and open in Google Colab.

Run the notebook, then read the instructions in the Moodle quiz. ONLY rerun the cells necessary to change the parameters indicated.